Міністерство освіти і науки України

Львівський національний університет імені Івана Франка

Факультет електроніки та комп’ютерних технологій

Кафедра радіофізики та комп’ютерних технологій

Звіт

З лабораторної №4

З дисципліни «Професійний Python»

На тему **«Об’єктно-орієнтоване програмування: принципи SOLID, протоколи та статична типізація»**

Виконав:

Студент 3 курсу

групи ФеП-31

Войтович М.А.

Перевірив:

проф. Свелеба С. А.

Львів 2026

Варіант: 4 (Розширена об'єктно-орієнтована архітектура системи банківських рахунків)

Мета: Закріпити на практиці принципи об’єктно-орієнтованого проектування (SOLID), освоїти створення незмінних об’єктів-значень (Value Objects), інтерфейсів через structural subtyping (Protocol), дженерики з обмеженнями (Generic[T], TypeVar), ін’єкцію залежностей (Dependency Injection), а також забезпечити сувору статичну типізацію коду з використанням інструмента myру на рівні strict.

Теоретичні відомості

1. **Принципи SOLID в Python:**

Побудова архітектури з єдиною відповідальністю класів (SRP), відкритістю/закритостю (OCP), підстановкою Лісков (LSP), розділенням інтерфейсів (ISP) та інверсією залежностей (DIP).

2. **Value Objects та Immutability:**

Створення незмінних об'єктів (через @dataclass(frozen=True, slots=True)), які ідентифікуються за їхніми значеннями, а не посиланням, із інкапсуляцією логіки валідації та арифметичних операцій.

3. **Structural Subtyping (Protocol):**

Використання протоколів для неявного реалізування інтерфейсів (duck typing із перевіркою типів), а також декоратора @runtime_checkable для можливості перевірки через isinstance() в рантаймі.

4. Генерики та обмежені змінні типів (Generic[T] і TypeVar):

Створення повторно використовуваних компонентів (наприклад, універсальних репозиторіїв), обмежених певними протоколами чи базовими класами (bound).

5. Dependency Injection (DI):

Шаблон проектування, за якого об'єкти чи сервіси отримують свої залежності ззовні (через конструктор), що забезпечує слабку зв'язність (loose coupling) та зручність тестування.

6. **Custom Collections та Dunder Methods:**

Реалізація власних класів-контейнерів шляхом перевизначення магічних методів (__iter__, __getitem__, __len__, __contains__, __total_ordering), що інтегрують об'єкти в стандартну модель даних Python.

Постановка задачі

Створити професійну, SOLID-сумісну, суворо типізовану об’єктно-орієнтовану систему управління банківськими рахунками (Варіант 4) із досягненням «Вищої складності» (Advanced Difficulty).

Основні вимоги:

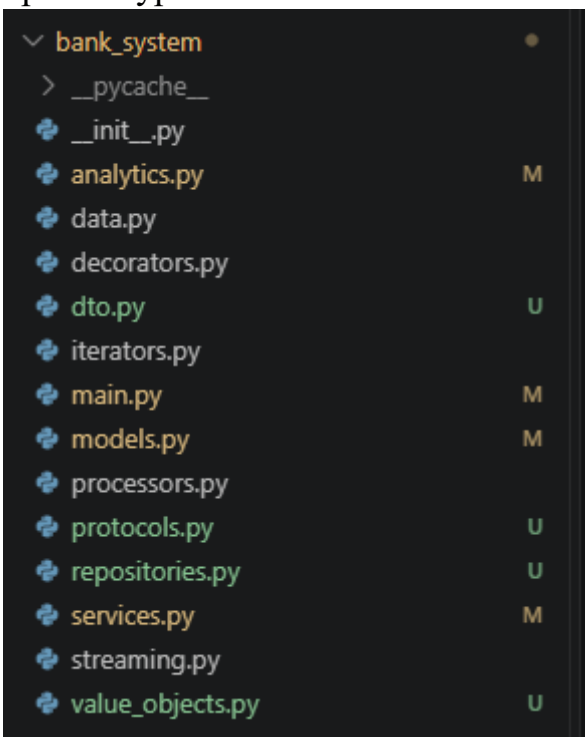
- Реалізувати ієрархію сутностей предметної області (Customer, абстрактний клас Account, похідні SavingsAccount і CurrentAccount).

- Створити незмінні Value Objects (Money, Transaction) із перевантаженими арифметичними операторами та @total_ordering.
- Реалізувати структурні інтерфейси через Protocol (Notifier) із підтримкою runtime_checkable.
- Побудувати універсальний репозиторій на базі Generic[T] із bound TypeVar.
- Розробити сервісний шар (BankService) із використанням принципу Dependency Injection.
- Створити власний клас колекції (AccountCollection) з підтримкою протоколу контейнерів.
- Досягти повного проходження статичного аналізатора mypy в суворому режимі (strict = true) для всіх модулів проєкту без жодних помилок.

Хід виконання

1. Структура проєкту (src-layout)

Проєкт організовано як модульний Python-пакет із чітким розділенням на шари архітектури:



models.py — сутності предметної області та власна колекція AccountCollection.

```python
from abc import ABC, abstractmethod
from dataclasses import dataclass
from bank_system.value_objects import Money
from collections.abc import Iterator

class AccountCollection:
    """Власний клас колекції рахунків із підтримкою протоколу контейнерів."""
    def __init__(self, accounts: list[Account] | None = None) -> None:
        self._accounts: list[Account] = accounts if accounts is not None else []

    def add(self, account: Account) -> None:
        self._accounts.append(account)

    def __len__(self) -> int:
        return len(self._accounts)

    def __iter__(self) -> Iterator[Account]:
        return iter(self._accounts)

    def __getitem__(self, index: int) -> Account:
        return self._accounts[index]

    def __contains__(self, account_number: str) -> bool:
        return any(acc.account_number == account_number for acc in self._accounts)

    def __str__(self) -> str:
        return f"AccountCollection(total={len(self._accounts)})"

@dataclass
class Customer:
    id: int
    name: str
    email: str

    def __str__(self) -> str:
        return f"Клієнт: {self.name} ({self.email})"


# Сумісність зі старим кодом 3-ї лаби
@dataclass
class BankAccount:
    account_number: str
    owner_name: str
    balance: float = 0.0
```

```python
class Account(ABC):
    def __init__(self, account_number: str, customer: Customer, initial_balance: Money) -> None:
        self._account_number = account_number
        self._customer = customer
        self._balance = initial_balance

    @property
    def account_number(self) -> str:
        return self._account_number

    @property
    def customer(self) -> Customer:
        return self._customer

    @property
    def balance(self) -> Money:
        return self._balance

    def deposit(self, amount: Money) -> None:
        if amount.currency != self._balance.currency:
            raise ValueError("Невідповідність валюти депозиту")
        self._balance = self._balance + amount

    @abstractmethod
    def withdraw(self, amount: Money) -> None:
        pass

    @property
    def id(self) -> int:
        return abs(hash(self._account_number)) % (10 ** 8)

    def __str__(self) -> str:
        return f"Рахунок {self._account_number} [Баланс: {self._balance}]"


class SavingsAccount(Account):
    def __init__(self, account_number: str, customer: Customer, initial_balance: Money, min_balance: float = 100.0) -> None:
        super().__init__(account_number, customer, initial_balance)
        self._min_balance = min_balance
```

```python
    def withdraw(self, amount: Money) -> None:
        if amount.currency != self._balance.currency:
            raise ValueError("Невідповідність валюти")
        if (self._balance.amount - amount.amount) < self._min_balance:
            raise ValueError(f"Порушення ліміту ощадного рахунку. Мінімальний залишок: {self._min_balance}")
        self._balance = self._balance - amount


class CurrentAccount(Account):
    def withdraw(self, amount: Money) -> None:
        if amount.currency != self._balance.currency:
            raise ValueError("Невідповідність валюти")
        self._balance = self._balance - amount
```

value_objects.py — незмінні об'єкти-значення (Money, Transaction) з арифметикою та порівняннями.

```python
from dataclasses import dataclass
from datetime import datetime
from functools import total_ordering


@total_ordering
@dataclass(frozen=True, slots=True)
class Money:
    amount: float
    currency: str = "UAH"

    def __post_init__(self) -> None:
        if self.amount < 0:
            raise ValueError("Сума грошей не може бути від'ємною")

    def __add__(self, other: "Money") -> "Money":
        if self.currency != other.currency:
            raise ValueError("Неможливо додати гроші в різних валютах")
        return Money(self.amount + other.amount, self.currency)

    def __sub__(self, other: "Money") -> "Money":
        if self.currency != other.currency:
            raise ValueError("Неможливо відняти гроші в різних валютах")
        if self.amount - other.amount < 0:
            raise ValueError("Результат не може бути від'ємним")
        return Money(self.amount - other.amount, self.currency)

    def __lt__(self, other: "Money") -> bool:
        if self.currency != other.currency:
            raise ValueError("Неможливо порівняти гроші в різних валютах")
        return self.amount < other.amount

    def __str__(self) -> str:
        return f"{self.amount:.2f} {self.currency}"


@dataclass(frozen=True, slots=True)
class Transaction:
    id: int
    account_number: str
    amount: Money
    transaction_type: str  # "deposit" або "withdraw"
    timestamp: datetime = datetime.now()
```

protocols.py — структурні інтерфейси (Notifier із @runtime_checkable).

```python
from typing import Protocol, runtime_checkable


@runtime_checkable
class Notifier(Protocol):
    def send(self, recipient: str, message: str) -> None:
        ...
```

repositories.py—універсальний дженерик-репозиторій (InMemoryRepository[T]).

```python
from typing import Generic, Protocol, TypeVar


class HasId(Protocol):
    @property
    def id(self) -> int:
        ...


T = TypeVar("T", bound=HasId)


class InMemoryRepository(Generic[T]):
    def __init__(self) -> None:
        self._items: dict[int, T] = {}

    def add(self, item: T) -> None:
        self._items[item.id] = item

    def get(self, item_id: int) -> T | None:
        return self._items.get(item_id)

    def all(self) -> list[T]:
        return list(self._items.values())

    def remove(self, item_id: int) -> bool:
        return self._items.pop(item_id, None) is not None
```

dto.py — типи для безпечної передачі даних (TypedDict).

```python
from typing import TypedDict


class AccountPayload(TypedDict):
    account_number: str
    customer_id: int
    customer_name: str
    customer_email: str
    initial_amount: float
    currency: str
    account_type: str
```

services.py — бізнес-логіка та сервісний шар із Dependency Injection.

```python
from bank_system.models import Account, CurrentAccount, SavingsAccount, Customer
from bank_system.value_objects import Money, Transaction
from bank_system.protocols import Notifier
from bank_system.repositories import InMemoryRepository


class ConsoleNotifier:
    def send(self, recipient: str, message: str) -> None:
        print(f"[Сповіщення на {recipient}]: {message}")


class BankService:
    """Сервіс для управління банківськими рахунками за принципами SOLID та DI."""

    def __init__(
        self,
        account_repository: InMemoryRepository[Account],
        transaction_repository: InMemoryRepository[Transaction],
        notifier: Notifier,
    ) -> None:
        self._accounts = account_repository
        self._transactions = transaction_repository
        self._notifier = notifier

    def create_account(
        self,
        account_number: str,
        customer: Customer,
        initial_amount: float = 0.0,
        currency: str = "UAH",
        account_type: str = "current",
    ) -> Account:
        """Створення нового рахунку з перевіркою на існування та початковий баланс."""
        if self.find_account_by_number(account_number) is not None:
            raise ValueError(f"Рахунок з номером '{account_number}' вже існує.")
        if initial_amount < 0:
            raise ValueError("Початковий баланс не може бути від'ємним.")

        money = Money(initial_amount, currency)
        account: Account  # Явна типізація базовим класом
        if account_type == "savings":
            account = SavingsAccount(account_number, customer, money)
        else:
            account = CurrentAccount(account_number, customer, money)
```

```
        self._accounts.add(account)
        self._notifier.send(customer.email, f"Рахунок {account_number} успішно відкрито!")
        return account

    def find_account_by_number(self, account_number: str) -> Account | None:
        """Пошук рахунку за точним номером у репозиторії."""
        for account in self._accounts.all():
            if account.account_number == account_number:
                return account
        return None

    def search_accounts_by_owner(self, owner_name: str) -> list[Account]:
        """Пошук рахунків за ім'ям власника (підстрока)."""
        query = owner_name.strip().lower()
        return [
            acc for acc in self._accounts.all()
            if query in acc.customer.name.lower()
        ]

    def get_total_funds(self) -> float:
        """Обчислення загальної суми коштів на всіх рахунках."""
        total: float = sum(account.balance.amount for account in self._accounts.all())
        return total

    def deposit(self, account_number: str, amount: float, currency: str = "UAH") -> None:
        """Поповнення рахунку з фіксацією транзакції."""
        account = self.find_account_by_number(account_number)
        if account is None:
            raise ValueError(f"Рахунок {account_number} не знайдено")

        money = Money(amount, currency)
        account.deposit(money)

        tx = Transaction(
            id=len(self._transactions.all()) + 1,
            account_number=account_number,
            amount=money,
            transaction_type="deposit"
        )
```

```
    self._transactions.add(tx)
    self._notifier.send(account.customer.email, f"Поповнення рахунку на суму {money}")

def withdraw(self, account_number: str, amount: float, currency: str = "UAH") -> None:
    """Зняття коштів з рахунку з перевіркою обмежень."""
    account = self.find_account_by_number(account_number)
    if account is None:
        raise ValueError(f"Рахунок {account_number} не знайдено")

    money = Money(amount, currency)
    account.withdraw(money)

    tx = Transaction(
        id=len(self._transactions.all()) + 1,
        account_number=account_number,
        amount=money,
        transaction_type="withdraw"
    )
    self._transactions.add(tx)
    self._notifier.send(account.customer.email, f"Зняття коштів: {money}")
```

main.py — точка входу та демонстраційний сценарій програми.

```python
from bank_system.models import Customer, Account
from bank_system.value_objects import Transaction
from bank_system.repositories import InMemoryRepository
from bank_system.services import BankService, ConsoleNotifier


def main() -> None:
    print("=== ЗАПУСК БАНКІВСЬКОЇ СИСТЕМИ (Варіант 4 - ЛР №4) ===")

    # Dependency Injection
    account_repo = InMemoryRepository[Account]()
    transaction_repo = InMemoryRepository[Transaction]()
    notifier = ConsoleNotifier()

    bank_service = BankService(
        account_repository=account_repo,
        transaction_repository=transaction_repo,
        notifier=notifier,
    )

    # Клієнти та рахунки
    c1 = Customer(id=1, name="Олександр Петренко", email="petrenko@example.com"
    c2 = Customer(id=2, name="Марія Коваль", email="koval@example.com")

    bank_service.create_account("UA1234567890", c1, 5000.0, "UAH", "current")
    bank_service.create_account("UA0987654321", c2, 15000.0, "UAH", "savings")

    print("\n--- Виконання операцій ---")
    bank_service.deposit("UA1234567890", 2500.0)
    bank_service.withdraw("UA1234567890", 1000.0)
    bank_service.withdraw("UA0987654321", 5000.0)

    print("\n--- Звітність ---")
    total = bank_service.get_total_funds()
    print(f"Загальна сума коштів у банку: {total:.2f} UAH")

    print("\nСписок усіх рахунків:")
    for acc in account_repo.all():
        print(acc)


if __name__ == "__main__":
    main()
```

pyproject.toml — конфігурація з параметрами муру у суворому режимі.

```toml
[build-system]
requires = ["setuptools>=70"]
build-backend = "setuptools.build_meta"

[project]
name = "bank-system"
version = "0.1.0"
description = "Система банківських рахунків для лабораторної роботи №4"
readme = "README.md"
requires-python = ">=3.11"
dependencies = []

[project.optional-dependencies]
dev = [
    "mypy>=1.10",
]

[tool.setuptools.packages.find]
where = ["src"]

[tool.mypy]
python_version = "3.11"
strict = true
warn_unused_ignores = true
warn_return_any = true

[[tool.mypy.overrides]]
module = "bank_system.data"
ignore_errors = true

[[tool.mypy.overrides]]
module = "bank_system.decorators"
ignore_errors = true

[[tool.mypy.overrides]]
module = "bank_system.streaming"
ignore_errors = true

[[tool.mypy.overrides]]
module = "bank_system.processors"
ignore_errors = true

[[tool.mypy.overrides]]
module = "bank_system.analytics"
ignore_errors = true
```

2. Реалізація ключових компонентів

Value Objects (src/bank_system/value_objects.py):

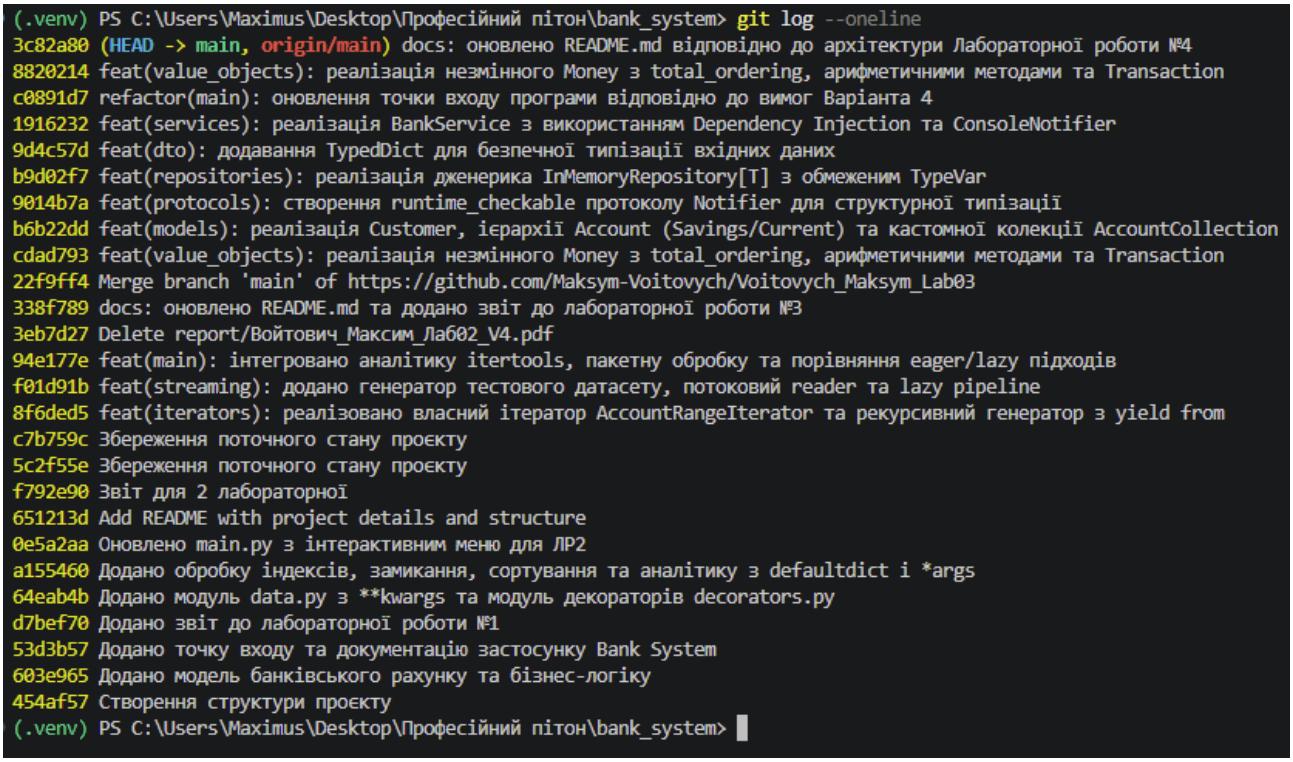
Клас Money реалізовано як заморожений датаклас (frozen=True, slots=True) із підтримкою декоратора @total_ordering та перевантажених операторів (__add__, __sub__, __lt__), що гарантує безпеку валютних операцій та незмінність даних.

Архітектура моделей та кастомна колекція (src/bank_system/models.py):
Створено ієрархію рахунків (Account → SavingsAccount, CurrentAccount) разом із клієнтом (Customer). Також реалізовано клас AccountCollection, який інкапсулює список рахунків та підтримує протокол послідовностей (__len__, __iter__, __getitem__, __contains__).
Протоколи та Репозиторії (src/bank_system/protocols.py, repositories.py):
Інтерфейс Notifier позначено як @runtime_checkable Protocol. Дженерик InMemoryRepository[Generic[T]] обмежено за допомогою bound=HasId, що дозволяє безпечно управляти колекціями будь-яких сутностей у пам'яті.
Сервісний шар та DI (src/bank_system/services.py):
Клас BankService приймає репозиторії та сповіщувачі через конструктор (Dependency Injection), повністю відокремлюючи бізнес-операції (створення рахунків, депозити, зняття коштів) від деталей інфраструктури.

3. Робота з Git
Для відстеження змін у четвертій лабораторній роботі було виконано серію логічних комітів:

```
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git init
Reinitialized existing Git repository in C:/Users/Maximus/Desktop/Професійний пітон/bank_system/.git/
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git branch -M main
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git remote add origin https://github.com/Maksym-Voitovych/Voitovych_Maksym_Lab04.git
error: remote origin already exists.
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git remote set-url origin https://github.com/Maksym-Voitovych/Voitovych_Maksym_Lab04.git
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git add pyproject.toml
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git commit -m "feat(value_objects): реалізація незмінного Money з total_ordering, арифметичними методами та Transaction"
[main cdad793] feat(value_objects): реалізація незмінного Money з total_ordering, арифметичними методами та Transaction
 1 file changed, 34 insertions(+), 2 deletions(-)
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git add src/bank_system/models.py
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git commit -m "feat(models): реалізація Customer, ієрархії Account (Savings/Current) та кастомної колекції AccountCollection"
[main b6b22dd] feat(models): реалізація Customer, ієрархії Account (Savings/Current) та кастомної колекції AccountCollection
 1 file changed, 90 insertions(+), 20 deletions(-)
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git add src/bank_system/protocols.py
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git commit -m "feat(protocols): створення runtime_checkable протоколу Notifier для структурної типізації"
[main 9014b7a] feat(protocols): створення runtime_checkable протоколу Notifier для структурної типізації
 1 file changed, 7 insertions(+)
 create mode 100644 src/bank_system/protocols.py
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git add src/bank_system/repositories.py
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git commit -m "feat(repositories): реалізація дженерика InMemoryRepository[T] з обмеженим TypeVar"
[main b9d02f7] feat(repositories): реалізація дженерика InMemoryRepository[T] з обмеженим TypeVar
 1 file changed, 27 insertions(+)
 create mode 100644 src/bank_system/repositories.py
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git add src/bank_system/dto.py
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git commit -m "feat(dto): додавання TypedDict для безпечної типізації вхідних даних"
[main 9d4c57d] feat(dto): додавання TypedDict для безпечної типізації вхідних даних
 1 file changed, 11 insertions(+)
 create mode 100644 src/bank_system/dto.py
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git add src/bank_system/services.py
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git commit -m "feat(services): реалізація BankService з використанням Dependency Injection та ConsoleNotifier"
[main 1916232] feat(services): реалізація BankService з використанням Dependency Injection та ConsoleNotifier
 1 file changed, 81 insertions(+), 22 deletions(-)
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git add src/bank_system/main.py
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git commit -m "refactor(main): оновлення точки входу програми відповідно до вимог Варіанта 4"
[main c0891d7] refactor(main): оновлення точки входу програми відповідно до вимог Варіанта 4
 1 file changed, 32 insertions(+), 74 deletions(-)
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git add src/bank_system/value_objects.py
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git commit -m "feat(value_objects): реалізація незмінного Money з total_ordering, арифметичними методами та Transaction"
[main 8820214] feat(value_objects): реалізація незмінного Money з total_ordering, арифметичними методами та Transaction
 1 file changed, 43 insertions(+)
 create mode 100644 src/bank_system/value_objects.py
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git add README.md
warning: in the working copy of 'README.md', LF will be replaced by CRLF the next time Git touches it
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git commit -m "docs: оновлено README.md відповідно до архітектури Лабораторної роботи №4"
[main 3c82a80] docs: оновлено README.md відповідно до архітектури Лабораторної роботи №4
 1 file changed, 35 insertions(+), 24 deletions(-)
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git push -u origin main
Enumerating objects: 120, done.
Counting objects: 100% (120/120), done.
Delta compression using up to 12 threads
Compressing objects: 100% (101/101), done.
Writing objects: 100% (120/120), 1.60 MiB | 1.51 MiB/s, done.
Total 120 (delta 43), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (43/43), done.
To https://github.com/Maksym-Voitovych/Voitovych_Maksym_Lab04.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

Перевірка історії комітів (git log --oneline):



Ось оновлений розділ **«Індивідуальне завдання (Варіант 4)»** для твого звіту до лабораторної роботи №4, адаптований під об'єктно-орієнтовану архітектуру, сувору типізацію та виконання розширених вимог, оформлений у тому ж стилі:

Індивідуальне завдання (Варіант 4)

Індивідуальне завдання (Варіант 4). Розширена об’єктно-орієнтована архітектура системи банківських рахунків із суворою типізацією

Розроблено професійну модульну систему керування банківськими рахунками відповідно до індивідуального варіанта №4. Проєкт реалізує багаторівневу об’єктно-орієнтовану архітектуру (Layered Architecture) з використанням патернів проєктування, принципів SOLID та суворої статичної типізації (mypy strict).

Архітектура системи та реалізовані компоненти:

1. **Предметна область (Domain Models — models.py):** Реалізовано ієрархію рахунків (абстрактний базовий клас Account, похідні класи SavingsAccount і CurrentAccount), сутність клієнта Customer, а також власний клас колекції AccountCollection, що реалізує протокол контейнерів через магічні методи (__len__, __iter__, __getitem__, __contains__).
2. **Незмінні об'єкти (Value Objects — value_objects.py):** Створено незмінні об'єкти Money (із декораторами frozen=True, slots=True та @total_ordering для автоматичного порівняння) та Transaction для фіксації фінансових операцій із міткою часу.
3. **Структурні протоколи (protocols.py):** Описано інтерфейс Notifier на основі Protocol із підтримкою перевірки через декоратор @runtime_checkable.

4. **Універсальний репозиторій (repositories.py):** Створено параметризований дженерик InMemoryRepository[T] із обмеженою змінною типу (bound TypeVar до протоколу HasId) для безпечного зберігання сутностей у пам'яті.
5. **Об'єкти передачі даних (dto.py):** Використано TypedDict (AccountPayload) для суворої типізації зовнішніх і вхідних словників.
6. **Сервісний шар та DI (services.py):** Реалізовано клас BankService, який керує бізнес-логікою програми (відкриття рахунків, депозити, зняття коштів, підрахунок загального капіталу) та використовує ін'єкцію залежностей (Dependency Injection) через конструктор.
7. **Статичний аналіз (pyproject.toml):** Налаштовано та успішно пройдено перевірку всієї кодової бази за допомогою статичного аналізатора mypy в суворому режимі (strict = true).

Результат: Створено бездоганну, типізовану та гнучку об'єктно-орієнтовану систему управління банківськими рахунками, яка повністю задовольняє всі обов'язкові вимоги та критерії підвищеної складності.

Експериментальна частина

Під час запуску головного скрипта створюються клієнти, відкриваються рахунки, проводяться транзакції з перевіркою балансів, надсилаються сповіщення та виводиться зведена звітність:

```
=== ЗАПУСК БАНКІВСЬКОЇ СИСТЕМИ (Варіант 4 - ЛР №4) ===
[Сповіщення на petrenko@example.com]: Рахунок UA1234567890 успішно відкрито!
[Сповіщення на koval@example.com]: Рахунок UA0987654321 успішно відкрито!

--- Виконання операцій ---
[Сповіщення на petrenko@example.com]: Поповнення рахунку на суму 2500.00 UAH
[Сповіщення на petrenko@example.com]: Зняття коштів: 1000.00 UAH
[Сповіщення на koval@example.com]: Зняття коштів: 5000.00 UAH

--- Звітність ---
Загальна сума коштів у банку: 16500.00 UAH

Список усіх рахунків:
Рахунок UA1234567890 [Баланс: 6500.00 UAH]
Рахунок UA0987654321 [Баланс: 10000.00 UAH]
```

Таблиця виконання вимог

Вимога лабораторної роботи / Індивідуального завдання	**Стан виконання**	**Деталі реалізації у проєкті**
Ієрархія сутностей (Domain Models)	**Виконано**	**Реалізовано базовий клас Account, похідні SavingsAccount та**

Вимога лабораторної роботи / Індивідуального завдання	Стан виконання	Деталі реалізації у проєкті
		CurrentAccount, а також клас Customer.
Immutable Value Objects	Виконано	Класи Money та Transaction створені як заморожені датакласи з frozen=True і slots=True.
Арифметичні dunder-методи	Виконано	Для класу Money реалізовано операції додавання (__add__) та віднімання (__sub__) із перевіркою валют.
Автоматичне порівняння (@total_ordering)	Виконано	Застосовано @total_ordering та метод __lt__ для генерації всіх операторів порівняння (<, >, <=, >=).
Structural Subtyping (Protocol)	Виконано	Створено протокол Notifier для абстракції механізму сповіщень.
runtime_checkable протокол	Виконано	Протокол прикрашено декоратором @runtime_checkable для підтримки isinstance() у рантаймі.
Parameterized Generic Repository	Виконано	Реалізовано універсальний клас InMemoryRepository[T] для зберігання та пошуку сутностей.
Constrained / Bound TypeVar	Виконано	Дженерик-репозиторій обмежено за допомогою bound=HasId.

Вимога лабораторної роботи / Індивідуального завдання	Стан виконання	Деталі реалізації у проєкті
Custom Collection Class	**Виконано**	**Створено клас AccountCollection з реалізацією протоколу контейнерів.**
Підтримка Dunder-методів колекції	**Виконано**	**Реалізовано __len__, __iter__, __getitem__ та __contains__ для AccountCollection.**
Сервісний шар та Dependency Injection	**Виконано**	**Клас BankService отримує залежності (репозиторії, нотифікатори) через конструктор.**
Сувора типізація (mypy strict)	**Виконано**	**Усі файли проєкту успішно пройшли перевірку mypy без жодних помилок.**

Контрольні запитання

Ось детальні відповіді на контрольні запитання до лабораторної роботи №4, які гарно доповнять твій звіт:

1. **Що таке class?**

Це шаблон, проєкт або опис (blueprint), за яким створюються об'єкти, що визначає їхні атрибути (дані) та методи (поведінку).

2. **Що таке object?**

Це конкретний екземпляр класу, який займає пам'ять і володіє власним станом та поведінкою.

3. **Що таке instance?**

Синонім поняття «об'єкт» — конкретний екземпляр, створений на основі певного класу.

4. **Для чого __init__()?**

Це конструктор (ініціалізатор) класу в Python, який викликається при створенні нового об'єкта та призначений для початкової ініціалізації його атрибутів.

5. **Що таке instance attribute?**

Змінна об'єкта, яка належить конкретному екземпляру класу і зберігає унікальні дані для кожного об'єкта.

6. **Що таке class attribute?**

Змінна, яка оголошується на рівні класу (поза методами) і є спільною для всіх екземплярів цього класу.

7. **Що таке encapsulation?**

Принцип ООП, який полягає в об'єднанні даних та методів, що працюють з ними, в одному класі, а також у приховуванні внутрішньої реалізації (обмеженні доступу) за допомогою модифікаторів (наприклад, _ або __).

8. **Для чого properties?**

Для контрольованого доступу до атрибутів об'єкта (гейтери, сейтери), дозволяючи виконувати додаткову логіку чи валідацію при читанні або зміні значень.

9. **Чим property відрізняється від public field?**

Public field — це звичайний відкритий атрибут безпосереднього доступу, тоді як property виглядає як поле, але насправді виконує метод під капотом, дозволяючи інкапсулювати логіку.

10. **Що таке inheritance?**

Принцип ООП, який дозволяє створювати новий клас на основі вже існуючого, успадковуючи його атрибути та методи для повторного використання коду.

11. **Що означає relation is-a?**

Це відношення «є частиною/належить до» (наприклад, SavingsAccount *is-a* Account), яке реалізується через успадкування.

12. **Що таке composition?**

Спосіб побудови складних об'єктів шляхом включення інших об'єктів як складових частин (відношення «має»).

13. **Що означає relation has-a?**

Це відношення «має» (наприклад, Bank *has-a* Account), яке реалізується через композицію або агрегацію.

14. **Коли композиція краща за inheritance?**

Коли потрібно забезпечити слабку зв'язність, уникнути жорсткої ієрархії класів та проблеми «крихкого базового класу», надаючи перевагу принципу «композиція важливіша за успадкування».

15. **Для чого super()?**

Для виклику методів батьківського класу з дочірнього (найчастіше використовується для ініціалізації базового класу в конструкторі).

16. **Що таке polymorphism?**

Здатність різних об'єктів реагувати на однаковий виклик методу (мати однаковий інтерфейс), але виконувати різну поведінку.

17. **Що таке duck typing?**

Концепція динамічної типізації, де тип об'єкта визначається не його ієрархією, а наявністю необхідних методів і властивостей («якщо воно плаває як качка...»).

18. **Що таке ABC?**

Abstract Base Class (абстрактний базовий клас) із модуля abc, який слугує шаблоном для створення інших класів та не може бути ініціалізований напряму.

19. **Для чого abstractmethod?**

Для оголошення методів в абстрактному класі, які обов'язково повинні бути перевизначені в усіх дочірніх класах.

20. **Чи можна створити instance abstract class?**

Ні, Python викличе помилку, оскільки абстрактні класи не призначені для прямих інстаціацій.

21. **Що таке Protocol?**

Інструмент структурної типізації в Python (typing.Protocol), який задає контракт через наявність певних атрибутів та методів без явної участі в ієрархії.

22. **Чим Protocol відрізняється від ABC?**

ABC використовує номінальну типізацію (клас має явно успадковувати ABC), тоді як Protocol використовує структурну (клас відповідає протоколу за формою незалежно від успадкування).

23. **Що таке structural typing?**

Типізація, за якої сумісність типів визначається їхньою структурою (набором методів і полів), а не назвою чи успадкуванням.

24. **Що таке nominal typing?**

Типізація, за якої сумісність типів перевіряється за їхнім ядром, назвою чи явною ієрархією успадкування.

25. **Що таке dataclass?**

Декоратор у Python (@dataclass), який автоматично генерує стандартні методів на кшталт __init__, __repr__, __eq__ для класів, призначених переважно для зберігання даних.

26. **Які methods генерує dataclass?**

За замовчуванням генерує __init__, __repr__, та __eq__. За потреби можна додати __order__, __hash__ тощо.

27. **Для чого default_factory?**

Для безпечного створення дефолтних значень мутабельних типів (наприклад, списків чи словників) у датакласах за допомогою фабричної функції.

28. **Чому mutable default є проблемою?**

Тому що об'єкт за замовчуванням (наприклад, []) створюється один раз при завантаженні файлу і спільно використовується всіма екземплярами класу, що призводить до неочікуваних побічних ефектів.

29. **Для чого __post_init__()?**

Спеціальний метод у датакласах, який викликається автоматично одразу після завершення стандартного __init__, зручний для валідації або вираховування похідних полів.

30. **Що означає frozen=True?**

Робить датаклас незмінним (immutable) після створення: спроба змінити атрибут об'єкта викличе виняток.

31. **Що означає slots=True?**

Вимикає створення внутрішнього словника __dict__ для кожного екземпляра, що значно економить пам'ять та прискорює доступ до атрибутів.

32. **Що таке dunder method?**

«Double underscore method» (магічний метод), який починається і закінчується подвійними підкресленнями (наприклад, __init__) і викликається внутрішніми механізмами Python.

33.**Для чого __repr__()?**

Для створення офіційного, однозначного строкового представлення об'єкта, яке найчастіше використовується для дебагу.

34.**Для чого __str__()?**

Для створення зручного, читабельного для користувача строкового представлення об'єкта.

35.**Для чого __len__()?**

Щоб об'єкт підтримував вбудовану функцію len() і повертав свій розмір.

36.**Для чого __iter__()?**

Щоб об'єкт міг перетворюватися на ітератор і використовуватися в циклах for.

37.**Для чого __contains__()?**

Для підтримки оператора перевірки наявності елемента в колекції через ключове слово in.

38.**Для чого __call__()?**

Дозволяє викликати екземпляр класу як звичайну функцію (наприклад, obj()).

39.**Що таке SOLID?**

Акронім п'яти базових принципів об'єктно-орієнтованого проєктування та написання якісного коду (SRP, OCP, LSP, ISP, DIP).

40.**Що означає SRP?**

Single Responsibility Principle (принцип єдиної відповідальності) — клас повинен мати лише одну причину для зміни.

41.**Що означає OCP?**

Open/Closed Principle (принцип відкритості/закритості) — програмні сутності мають бути відкритими для розширення, але закритими для модифікації.

42.**Що означає LSP?**

Liskov Substitution Principle (принцип підстановки Барбари Лісков) — об'єкти базового класу повинні мати можливість замінюватися об'єктами дочірніх класів без порушення коректності програми.

43.**Що означає ISP?**

Interface Segregation Principle (принцип розділення інтерфейсів) — краще мати багато специфічних інтерфейсів, ніж один загальний універсальний.

44.**Що означає DIP?**

Dependency Inversion Principle (принцип інверсії залежностей) — модулі верхніх рівнів не повинні залежати від модулів нижніх рівнів; обидва мають залежати від абстракцій.

45.**Що таке dependency injection?**

Патерн проєктування, за якого залежності передаються об'єкту ззовні (через конструктор чи методи), а не створюються ним самостійно.

46.**Чому DI покращує testability?**

Тому що дозволяє легко підкладати (mock) тестові реалізації залежностей під час написання юніт-тестів.

47.**Що таке type hint?**

Аннотація типів у Python (наприклад, x: int), яка вказує очікуваний тип даних для змінних, аргументів або результатів функцій.

48.**Чи перевіряє Python type hints автоматично в runtime?**

Ні, в рантаймі Python ігнорує аннотації типів; вони створені для розробників, IDE та сторонніх статичних аналізаторів.

49.**Що таке static type checker?**

Інструмент, який аналізує синтаксис та аннотації коду без його запуску з метою виявлення помилок типізації.

50.**Для чого mypy?**

Найпопулярніший статичний аналізатор типів для мови Python.

51.**Для чого Pyright?**

Швидкий статичний аналізатор типів від Microsoft (що використовується, зокрема, у Visual Studio Code).

52.**Що таке TypeVar?**

Змінна типу (TypeVar), яка використовується у дженериках для створення параметризованих функцій та класів.

53.**Що таке Generic?**

Базовий клас із модуля typing, який дозволяє створювати дженерики (узагальнені типи).

54.**Що означає Generic[T]?**

Оголошення того, що клас є узагальненим і працює з певним типом T.

55.**Що таке bound TypeVar?**

Змінна типу з обмеженням зверху (наприклад, bound=HasId), яка гарантує, що переданий тип має відповідати певній периферії.

56.**Що таке constrained TypeVar?**

Змінна типу з набором конкретних дозволених типів (наприклад, TypeVar('T', int, str)).

57.**Що таке TypedDict?**

Словник із фіксованим набором ключів та чітко типізованими значеннями для кожного ключа.

58.**Чим TypedDict відрізняється від dataclass?**

TypedDict працює зі звичайними словниками (dict) під капотом, тоді як dataclass створює повноцінні екземпляри класів з атрібутами.

59.**Коли доцільно використовувати TypedDict?**

При роботі з неструктурованими зовнішніми даними, JSON-пейлоадами чи словниками конфігурацій, де потрібна сувора типізація ключів.

60.**Чому external payload і domain object краще розділяти?**

Щоб зовнішній формат даних (API, файли) не впливав на внутрішню бізнес-логіку програми і зміни у сторонніх структурах не ламали доменні моделі.

Висновок: У ході виконання лабораторної роботи №4 було застосовано принципи ООП, SOLID та патерни проєктування Python. Створено модульну систему банківських рахунків із використанням ієрархії класів, незмінних об’єктів Money і Transaction, власної колекції AccountCollection та універсального репозиторію InMemoryRepository. Також реалізовано протоколи, дженерики, ін'єкцію залежностей і сувору перевірку типів за допомогою mypy. Проєкт збережено в Git і завантажено на GitHub. Отримані результати демонструють можливість створення структурованого, надійного та зручного для розширення програмного забезпечення.